Ex. No. 6	Data Persistence Using SQLite
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To design and develop a Flutter application that stores and retrieves student records using an SQLite database by implementing Create and Read operations.

Description

Data persistence is the process of storing application data so that it remains available even after the application is closed or refreshed. SQLite is a lightweight relational database that stores information in tables. The sqflite package provides SQLite database operations in Flutter. Since this experiment is executed in Google Chrome, the sqflite_common_ffi_web package is used to enable SQLite support on the web platform.

The application is named StudentHub. It allows the user to enter a student's name, register number, department and email address. When the Save Student button is pressed, the application validates the entered details and inserts the record into the students table. The application then retrieves the saved records and displays them in the Student Records section.

The students table contains the following fields:
• id – INTEGER PRIMARY KEY AUTOINCREMENT
• name – TEXT NOT NULL
• registerNumber – TEXT NOT NULL
• department – TEXT NOT NULL
• email – TEXT NOT NULL

The main concepts used are DatabaseHelper for database initialization and operations, CREATE TABLE for creating the students table, INSERT for storing student details, SELECT for retrieving records, the Student model for converting data between objects and maps, and StatefulWidget with setState() for updating the interface.

The application was executed in Google Chrome at localhost:5555. The output screenshots show the student registration form and a saved student record displayed in the Student Records list.

Program

```
import 'package:flutter/material.dart';
import 'package:flutter/foundation.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';
import 'package:path/path.dart' as p;

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  if (kIsWeb) {
    databaseFactory = databaseFactoryFfiWeb;
  }
  runApp(const StudentHubApp());
}

// STUDENT MODEL
class Student {
  final int? id;
  final String name, registerNumber, department, email;
```

```
const Student({
  this.id,
  required this.name,
  required this.registerNumber,
  required this.department,
  required this.email,
});

Map<String, dynamic> toMap() => {
  'name': name,
  'registerNumber': registerNumber,
  'department': department,
  'email': email,
};

factory Student.fromMap(Map<String, dynamic> map) => Student(
  id: map['id'] as int,
  name: map['name'] as String,
  registerNumber: map['registerNumber'] as String,
  department: map['department'] as String,
  email: map['email'] as String,
);
```

```
}

// DATABASE HELPER
class DatabaseHelper {
  DatabaseHelper._();
  static final DatabaseHelper instance = DatabaseHelper._();
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    final String databasePath;
    if (kIsWeb) {
      databasePath = 'studenthub.db';
    } else {
      final path = await getDatabasesPath();
      databasePath = p.join(path, 'studenthub.db');
    }
```

```
  return openDatabase(
    databasePath,
    version: 1,
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE students (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT NOT NULL,
          registerNumber TEXT NOT NULL,
          department TEXT NOT NULL,
          email TEXT NOT NULL
        )
      ''');
    },
  );
}

// CREATE operation
Future<int> insertStudent(Student student) async {
  final db = await database;
  return db.insert('students', student.toMap(),
      conflictAlgorithm: ConflictAlgorithm.abort);
}
```

```
  // READ operation
  Future<List<Student>> getStudents() async {
    final db = await database;
    final result = await db.query('students', orderBy: 'id DESC');
    return result.map((map) => Student.fromMap(map)).toList();
  }
}

// MAIN APPLICATION
class StudentHubApp extends StatelessWidget {
  const StudentHubApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'StudentHub',
      theme: ThemeData(
        colorSchemeSeed: const Color(0xFF4053B5),
        useMaterial3: true,
      ),
      home: const StudentHomePage(),
```

```
    );
  }
}

// STUDENT HOME PAGE
class StudentHomePage extends StatefulWidget {
  const StudentHomePage({super.key});

  @override
  State<StudentHomePage> createState() => _StudentHomePageState();
}

class _StudentHomePageState extends State<StudentHomePage> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _registerController = TextEditingController();
  final _departmentController = TextEditingController();
  final _emailController = TextEditingController();

  List<Student> students = [];
  bool isLoading = true;
  bool isSaving = false;
```

```
@override
void initState() {
  super.initState();
  loadStudents();
}

// READ saved records
Future<void> loadStudents() async {
  final records = await DatabaseHelper.instance.getStudents();
  if (!mounted) return;
  setState(() {
    students = records;
    isLoading = false;
  });
}

// CREATE a new record
Future<void> saveStudent() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => isSaving = true);

  final student = Student(
    name: _nameController.text.trim(),
```

```
    registerNumber: _registerController.text.trim(),
    department: _departmentController.text.trim(),
    email: _emailController.text.trim(),
  );

  try {
    await DatabaseHelper.instance.insertStudent(student);
    _nameController.clear();
    _registerController.clear();
    _departmentController.clear();
    _emailController.clear();
    await loadStudents();

    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Student saved successfully!')),
    );
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Unable to save student: $e')),
    );
  } finally {
```

```
    if (mounted) setState(() => isSaving = false);
  }
}

String? requiredValidator(String? value) {
  if (value == null || value.trim().isEmpty) {
    return 'This field is required';
  }
  return null;
}

@override
void dispose() {
  _nameController.dispose();
  _registerController.dispose();
  _departmentController.dispose();
  _emailController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
```

```
backgroundColor: const Color(0xFFF4F5FA),
appBar: AppBar(
  title: const Text('StudentHub',
      style: TextStyle(fontWeight: FontWeight.bold)),
  backgroundColor: const Color(0xFF4053B5),
  foregroundColor: Colors.white,
  actions: [
    IconButton(
      icon: const Icon(Icons.refresh),
      onPressed: loadStudents,
    ),
  ],
),
body: Center(
  child: ConstrainedBox(
    constraints: const BoxConstraints(maxWidth: 750),
    child: ListView(
      padding: const EdgeInsets.all(16),
      children: [
        Container(
          padding: const EdgeInsets.all(20),
          decoration: BoxDecoration(
            color: const Color(0xFFE8EAF6),
```

```
    borderRadius: BorderRadius.circular(16),
  ),
  child: const Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text('Student Record Management',
          style: TextStyle(fontSize: 21,
              fontWeight: FontWeight.bold,
              color: Color(0xFF4053B5))),
      SizedBox(height: 6),
      Text('Add and manage student records using SQLite database.'),
    ],
  ),
),
const SizedBox(height: 22),
Card(
  color: Colors.white,
  elevation: 3,
  child: Padding(
    padding: const EdgeInsets.all(18),
    child: Form(
      key: _formKey,
      child: Column(
```

```
crossAxisAlignment: CrossAxisAlignment.start,
children: [
 const Text('Add New Student',
    style: TextStyle(fontSize: 20,
       fontWeight: FontWeight.bold)),
 const SizedBox(height: 18),
 _buildField('Student Name', 'Enter student name',
    Icons.person, _nameController),
 _buildField('Register Number', 'Enter register number',
    Icons.badge, _registerController),
 _buildField('Department', 'Enter department',
    Icons.school, _departmentController),
 _buildField('Email Address', 'Enter email address',
    Icons.email, _emailController, email: true),
 const SizedBox(height: 8),
 SizedBox(
  width: double.infinity,
  height: 48,
  child: ElevatedButton.icon(
   onPressed: isSaving ? null : saveStudent,
   icon: const Icon(Icons.save),
   label: Text(isSaving ? 'Saving...' : 'Save Student'),
   style: ElevatedButton.styleFrom(
```

```
                backgroundColor: const Color(0xFF4053B5),
                foregroundColor: Colors.white,
              ),
            ),
          ),
        ],
      ),
    ),
  ),
),
const SizedBox(height: 24),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    const Text('Student Records',
        style: TextStyle(fontSize: 20,
            fontWeight: FontWeight.bold)),
    Chip(label: Text('${students.length} Students')),
  ],
),
const SizedBox(height: 8),
if (isLoading)
  const Center(child: CircularProgressIndicator())
```

```
          else if (students.isEmpty)
            const Card(
              child: Padding(
                padding: EdgeInsets.all(24),
                child: Center(child: Text('No student records found.')),
              ),
            )
          else
            ...students.map((student) => StudentCard(student: student)),
        ],
      ),
    ),
  ),
 );
}

Widget _buildField(String label, String hint, IconData icon,
    TextEditingController controller, {bool email = false}) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 14),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
```

```
Text(label, style: const TextStyle(fontWeight: FontWeight.w600)),
const SizedBox(height: 7),
TextFormField(
  controller: controller,
  validator: (value) {
    final error = requiredValidator(value);
    if (error != null) return error;
    if (email &&
        !RegExp(r'^[^@]+@[^@]+\.[^@]+$').hasMatch(value!.trim())) {
      return 'Enter a valid email address';
    }
    return null;
  },
  decoration: InputDecoration(
    hintText: hint,
    prefixIcon: Icon(icon, color: const Color(0xFF4053B5)),
    filled: true,
    fillColor: const Color(0xFFF7F7FB),
    border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(11)),
  ),
 ),
],
```

```
      ),
    );
  }
}

// REUSABLE STUDENT CARD
class StudentCard extends StatelessWidget {
  final Student student;
  const StudentCard({super.key, required this.student});

  @override
  Widget build(BuildContext context) {
    return Card(
      color: Colors.white,
      elevation: 2,
      margin: const EdgeInsets.only(bottom: 12),
      child: ListTile(
        contentPadding: const EdgeInsets.all(14),
        leading: CircleAvatar(
          backgroundColor: const Color(0xFFC5CAE9),
          child: Text(student.name.isNotEmpty
              ? student.name[0].toUpperCase() : '?',
              style: const TextStyle(color: Color(0xFF4053B5),
```

```
            fontWeight: FontWeight.bold)),
      ),
      title: Text(student.name,
        style: const TextStyle(fontWeight: FontWeight.bold)),
      subtitle: Padding(
       padding: const EdgeInsets.only(top: 6),
       child: Text(
        'Register No: ${student.registerNumber}\n'
        'Department: ${student.department}\n'
        'Email: ${student.email}',
       ),
      ),
      trailing: const Icon(Icons.storage, color: Colors.green),
    ),
   );
  }
}
```

Output Screenshots

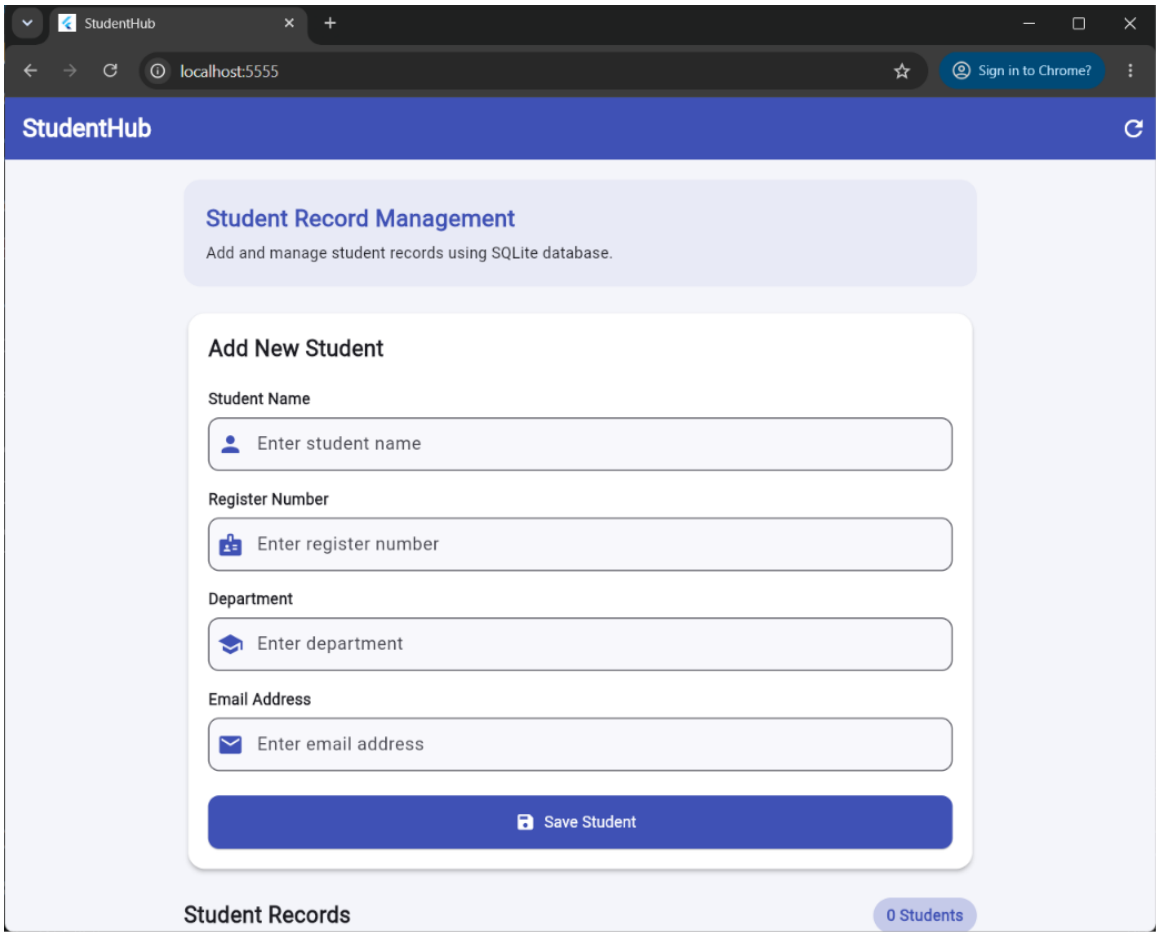


Figure 1: StudentHub application displaying the student registration form.

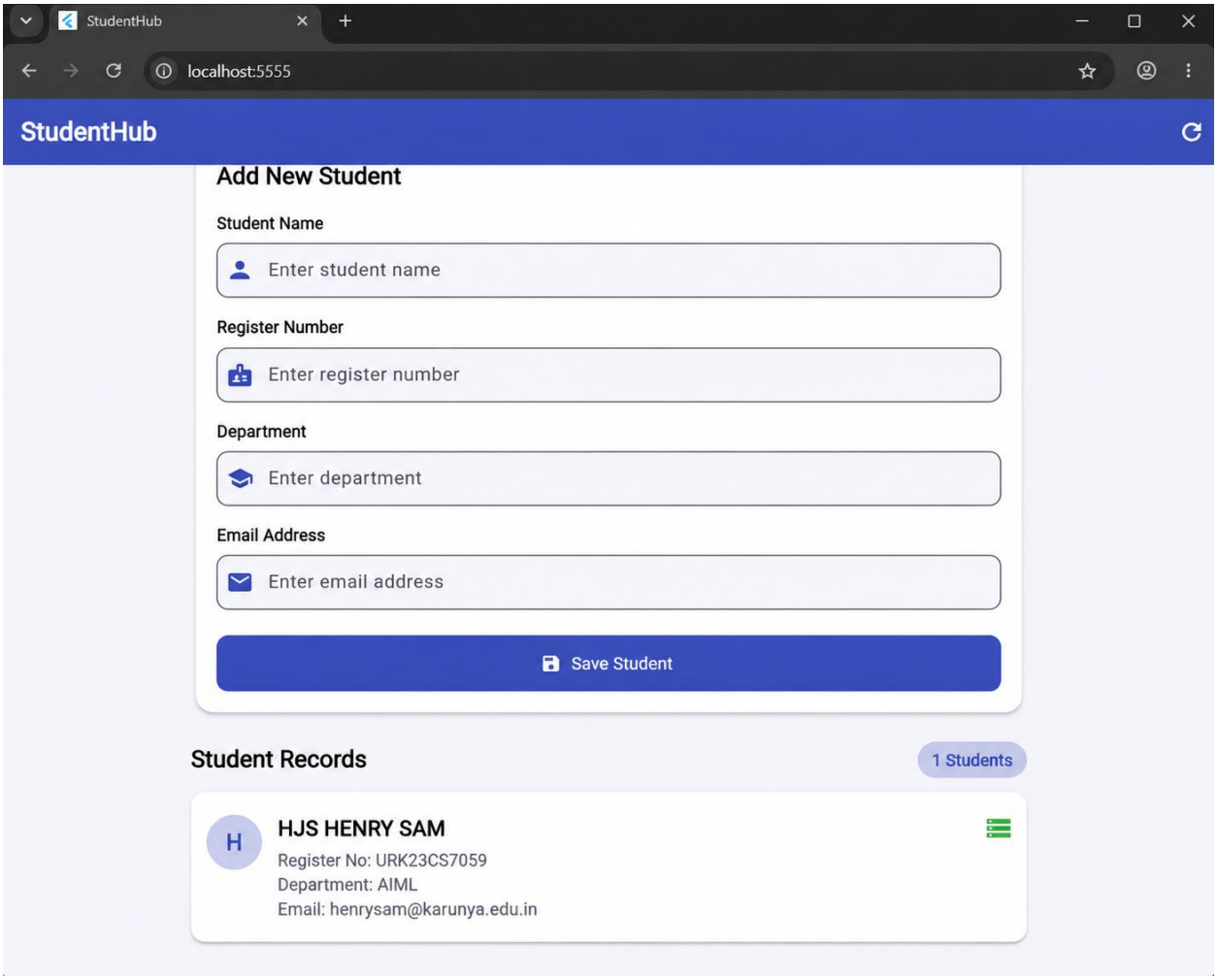


Figure 2: Student record successfully saved and displayed in the Student Records list.

Result

Thus, the Flutter StudentHub application was successfully developed to perform Create and Read operations using an SQLite database. Student details entered through the form were inserted into the students table and retrieved from the database for display in the application. The application was executed successfully in Google Chrome using SQLite Web support.